Quick Reference



CHOOSE AND INSPECT

I need...	Use
Constant shared by all objects	class attribute
Per-object state	instance attribute set in `__init__`
Validated or computed attribute	`@property` (+ setter)
Alternative constructor / class-wide logic	`@classmethod`
Helper that needs no object or class	`@staticmethod`
Specialise a true subtype	inheritance + `super()`
Assemble behaviours, swap parts	composition + delegation
Enforce an interface	`ABC` + `@abstractmethod`
Plain data container	`@dataclass`

Introspection helper	What it gives you
`type(o)`	class of `o`
`isinstance(o, C)`	`o` is a `C` (or subclass)
`issubclass(S, C)`	class `S` derives from `C`
`hasattr(o, "x")`	does attribute exist?
`getattr(o, "x", d)`	read, with default
`setattr` / `delattr`	write / delete by name
`vars(o)` / `o.__dict__`	instance namespace
`dir(o)`	all visible attribute names
`C.__mro__`, `C.__bases__`	resolution order, direct bases

SPECIAL METHODS

Special (dunder) method	Triggered by
`__init__`, `__new__`	initialise, create
`__repr__`, `__str__`	debug text, user text
`__eq__`, `__lt__`, `__hash__`	`==`, `<`, hashing
`__add__`, `__mul__`	`+`, `*`
`__len__`, `__iter__`	`len(o)`, `for x in o`
`__getitem__`	`o[key]`
`__call__`	`o(...)`
`__enter__`, `__exit__`	`with o:`
`__bool__`, `__contains__`	`bool(o)`, `x in o`

TOP PITFALLS

- **Mutable class attribute or default** (`[]`, `{}`) is shared; create it in `__init__` or use `None`.
- **Forgot** `self` in the signature, or used a bare name instead of `self.name`.
- `self.n += 1` **on a class attribute** shadows it; use `type(self).n`.
- **Calling a base class by name** in multiple inheritance; use `super()`.
- **Inheriting just to reuse code** where *has-a* fits; compose instead.
- `@staticmethod` **where** `cls` **is needed** breaks subclasses; use `@classmethod`.
- **Property storing into its own name** (`self.x = v` inside the `x` setter) recurses forever; store in `self._x`.
- **Overriding** `__eq__` **without** `__hash__` makes instances unhashable.